

Trabajo Práctico Integrador

Estructuras de Datos, Contrato Equals/HashCode y Complejidad Algorítmica

Integrantes 3k10

Lautaro Ellerbach

Joaquin Velazquez

Donato Fornasari

Serena Sosa

Agusto De Giuseppe

Juan Manuel Aravena

Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria

Responder y Justificar:

1. Trazo del Estado Interno: Explique paso a paso qué ocurre dentro de la estructura HashSet:

- o Al ejecutar `tarjetasValidas.add(t1)`
- o Al ejecutar `t1.setNivelAcceso(5)`.
- o Al ejecutar `tarjetasValidas.contains(t1)`.

2. Explicación del Cajón (Bucket): ¿En qué número/ubicación de cajón quedó guardado el objeto y en cuál lo busca Java al hacer `contains()`?

3. Fuga de Memoria (Memory Leak): ¿Por qué este patrón genera una fuga de memoria si se ejecuta repetidamente en un servidor que opera 24/7?

4. Refactorización Obligatoria: Aplique dos soluciones arquitectónicas distintas para corregir el problema (una basada en inmutabilidad y otra ajustando el criterio de la entidad).

Respuestas:

1.

- Al ejecutar `tarjetasValidas.add(t1)`: Java invoca el método `hashCode()` del objeto `t1`, el cual calcula un valor entero utilizando el código ("ACC-101") y el `nivelAcceso` (2). Con este valor, la estructura determina un índice específico (una dirección de memoria del heap, Bucket) y guarda la referencia del objeto allí.
- Al ejecutar `t1.setNivelAcceso(5)`: Se muta el estado interno del objeto. Como el `nivelAcceso` es parte del cálculo del `hashCode()`, anteriormente 2, el código hash de la tarjeta `t1` cambia automáticamente. Sin embargo, el HashSet no cambia porque este no realiza una actualización cuando un estado interno cambia, por lo que se debe volver a llamar para lograr que físicamente el objeto se guarde en un Bucket distinto.
- Al ejecutar `tarjetasValidas.contains(t1)`: Vuelve a invocar `hashCode()` sobre `t1` para saber dónde buscar. Ahora, el cálculo se hace con el `nivelAcceso` igual a 5, generando un código hash completamente distinto. El HashSet se dirige al nuevo Bucket calculado; al encontrarlo vacío, retorna `false`.

2. El Bucket en el que quedó guardado el objeto es el correspondiente al cálculo entre el código ("ACC-101") y el `nivelAcceso` (2). El cual discrepa del obtenido en la búsqueda con `contains()`, ya que este es calculado con el código ("ACC-101") y el `nivelAcceso` (5). Debido a que no coinciden la búsqueda falla porque retorna vacío.

3. El problema nace debido a que en Java existe un programa automático llamado Garbage Collector (Recolector de Basura), el cual recorre la memoria Heap en búsqueda de objetos que no se estén usando, estos son los carentes de referencia (ninguna variable ni colección los apunta).

Para ilustrar el problema usare de ejemplo el código de este ejercicio, cuando hacemos `tarjetasValidas.add(t1)` con código ACC-101 y `nivelAcceso` 2, el HashSet calcula un Bucket ,

supongamos el Bucket 8. En ese momento, el HashSet tiene una referencia que lo ata a la tarjeta t1. Mientras esta referencia exista el Garbage Collector no lo borra.

Ahora le cambiamos un estado interno con `t1.setNivelAcceso(5)`, su código Hash cambia pero físicamente el Bucket sigue siendo el 8 porque no hemos llamado nuevamente a HashSet.

En el caso que queramos remover la tarjeta porque un empleado se va de la empresa con `tarjetasValidas.remove(t1)`. Calcula un nuevo código Hash con el nivelAcceso 5, supongamos el Bucket 16, este está vacío por lo que Java no borra nada.

El problema surge porque ese remove no se efectuó y la tarjeta sigue guardada físicamente en el Bucket 8, como la tarjeta aún está dentro del HashSet el Garbage Collector no la elimina.

Si esto sucediera en un servidor de una empresa que funciona 24/7 (es decir la memoria Heap no se libera), y se cambian diariamente el nivel de acceso a 100 empleados, tendríamos 100 tarjetas imposibles de remover por día. En unos meses el sistema colapsara porque se llenó por completo la memoria RAM del servidor.

4. Para evitar la Refactorización Obligatoria aplicaremos dos enfoques distintos:

- **Solución basada en Inmutabilidad:** Consta en declarar que una clase sea completamente inmutable, para ello declaramos los atributos como `private final` y eliminando el método `setnivelAcceso`. Si el empleado cambia de nivel, se debe crear una nueva instancia de `TarjetaAcceso`. Esto nos otorga la certeza de que tanto `contains()` como `remove()` calculen el `HashCode` sea el indicado. Código aplicando esta solución:

```
final class TarjetaAcceso {
    private final String codigo;
    private final int nivelAcceso;

    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }

    public String getCodigo() { return codigo; }
    public int getNivelAcceso() { return nivelAcceso; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        TarjetaAcceso tarjeta = (TarjetaAcceso) o;
        return nivelAcceso == tarjeta.nivelAcceso && Objects.equals(codigo,
            tarjeta.codigo);
    }
}
```

```

        @Override
        public int hashCode() {
            return Objects.hash(codigo, nivelAcceso);
        }
    }
}

```

- **Solución basada en el ajuste del criterio de Entidad:** Si las reglas de negocio exigen que el nivel de acceso sea mutable, los métodos equals() y hashCode() deben calcularse únicamente sobre un identificador inmutable (en nuestro caso, el Código). De esta manera no importara cuantas veces cambie el nivel de acceso, el hashCode permanecerá constante y el objeto siempre se encontrará en el mismo Bucket. Se implementa cambiando el **return Objects.hash (codigo, nivelAcceso)** por **return Objects.Hash(codigo)**. Código aplicando esta solución:

```

class TarjetaAcceso {
    private final String codigo; // Identificador inmutable
    private int nivelAcceso;     // Atributo mutable

    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }

    public void setNivelAcceso(int nivelAcceso) {
        this.nivelAcceso = nivelAcceso;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        TarjetaAcceso tarjeta = (TarjetaAcceso) o;
        return Objects.equals(codigo, tarjeta.codigo); // Criterio de identidad único
    }

    @Override
    public int hashCode() {
        return Objects.hash(codigo); // Hash constante independientemente del nivel
    }
}

```

Consigna 2: Selección y Diseño de Arquitectura de Colecciones

Responder y Justificar:

Requerimiento 1: Sistema de Auditoría de Transacciones

Registrar operaciones en el orden exacto en que ocurren. Se requiere insertar miles de registros por segundo al final de la colección y recorrerlos secuencialmente.

Requerimiento 2: Catálogo de Ofertas Relámpago

Mantener los productos ordenados automáticamente por precio de menor a mayor.

Debe permitir consultar eficientemente el producto más barato y el más caro.

Requerimiento 3: Búsqueda de Sesiones de Usuarios Activos

Buscar instantáneamente la sesión de un usuario dado su Token (String único) entre 500,000 usuarios conectados en simultáneo.

Completar la siguiente tabla comparativa y justificar la decisión con la complejidad algorítmica Big O:

Requerimiento	Interfaz Seleccionada	Implementación Concreta	Complejidad Búsqueda	Complejidad Inserción	Justificación Técnica de la Elección
Auditoría	List	ArrayList	$O(n)$	$O(1)$	Mantener el orden exacto de inserción y permitir duplicados. Se usa ArrayList porque la inserción es al final del arreglo y el recorrido es secuencial en memoria. Son operaciones muy rápidas y eficientes.
Ofertas	Set	TreeSet	$O(\log n)$	$O(\log n)$	Rechaza duplicados y mantiene un ordenamiento ascendente por precio. TreeSet inserta y busca con la misma complejidad, sus métodos permiten obtener los extremos.
Sesiones	Map	HashMap	$O(1)$	$O(1)$	Estructura ideal para asociar pares de tokens sin importar el orden de ingreso. Ante 500,000 usuarios conectados en simultáneo, hashMap es la opción más veloz porque localiza los datos usando el hash en tiempo constante

Consigna 3: Optimización Algorítmica ($O(n^2)$ -> $O(n)$)

Un desarrollador escribió el siguiente método para encontrar el primer carácter que no se repite dentro de una cadena de texto (ejemplo: en "barco", el primero es 'b'; en "abarca", el primero es 'r').

```
public static Character primerCaracterNoRepetidoLento(String texto) { // Algoritmo
ineficiente O(n^2) for (int i = 0; i < texto.length(); i++) { char c = texto.charAt(i); boolean
duplicado = false; for (int j = 0; j < texto.length(); j++) { if (i != j && c == texto.charAt(j)) {
duplicado = true; break; } } if (!duplicado) return c; } return null; }
```

Responder y Justificar:

1. Refactorización a O(n): Escriba una versión optimizada del método utilizando la colección adecuada de java.util (HashMap, LinkedHashMap, o HashSet) que resuelva el problema realizando como máximo dos recorridos lineales (O(n)).

2. Análisis de Colisiones: Si se utiliza un HashMap internamente, explique qué ocurre si dos caracteres distintos generan el mismo hashCode() (Colisión de Hash) y cómo la estructura resuelve este problema sin perder información.

Respuestas:

1.

```
import java.util.HashMap;
import java.util.Map;

public static Character primerCaracterNoRepetidoRapido(String texto) {
    if (texto == null || texto.isEmpty()) {
        return null;
    }

    // Primer Recorrido
    Map<Character, Integer> conteo = new HashMap<>();
    for (int i = 0; i < texto.length(); i++) {
        char c = texto.charAt(i);
        conteo.put(c, conteo.getOrDefault(c, 0) + 1);
    }

    // Segundo recorrido
    for (int i = 0; i < texto.length(); i++) {
        char c = texto.charAt(i);
        if (conteo.get(c) == 1) {
            return c;
        }
    }

    return null;
}
```

2. Un HashMap almacena internamente un arreglo de *buckets*. La posición del bucket se calcula aplicando una función hash sobre el hashCode () de la clave. Cuando dos claves distintas caen en el mismo bucket (colisión), Java **no sobrescribe** el dato existente. En su lugar, dentro de ese bucket se mantiene una **lista enlazada** de entradas (Entry objects, pares clave-valor). La nueva entrada se agrega a esa lista. A esto último se le llama Encadenamiento (Separate Chaining). Cuando se busca o inserta una clave (por ejemplo, con get () o put ()), el HashMap:

- Calcula el `hashCode()` para ubicar el bucket correcto.
- Si en ese bucket hay múltiples entradas (por colisión), recorre la lista/árbol y usa el método `equals()` para comparar la clave buscada contra cada clave existente en el bucket, hasta encontrar la coincidencia exacta.
- `hashCode()` solo sirve para **ubicar el bucket rápidamente**, pero `equals()` es quien garantiza que **no se confundan** dos claves distintas que colisionaron. Así, aunque 'a' y 'b' (hipotéticamente) generarán el mismo `hashCode()`, sus valores char son distintos y `equals()` los distinguiría correctamente, manteniendo ambas entradas separadas sin sobrescribir ni perder información.

Consigna 4: Caso Teórico de Borde y Colisiones

Considere las siguientes dos instancias de String: `String s1 = "FB";` `String s2 = "Ea";`
Sabiendo que `s1.hashCode() == s2.hashCode()` devuelve true (ambos dan el entero 2236):

1. Si guardamos ambas cadenas en un `HashSet`, ¿cuántos elementos contendrá la colección? Explique cómo utiliza Java los métodos `hashCode()` y `equals()` internamente para determinar si `s2` debe ingresarse o desecharse.
2. ¿Cómo afecta la acumulación de colisiones de hash al rendimiento teórico de un `HashMap` en el peor escenario posible (de $O(1)$ a $O(n)$ o $O(\log n)$)?

Respuestas:

1. **El `HashSet` contendrá 2 elementos.**

Aunque "FB" y "Ea" tengan el **mismo `hashCode()` (2236)**, esto **no las convierte en "iguales"** para Java. Un `HashSet` internamente utiliza un `HashMap` (donde los elementos del Set son las claves), por lo que el proceso de inserción sigue esta lógica:

Paso a paso al insertar `s2 = "Ea"` (después de haber insertado `s1 = "FB"`):

- **Cálculo del bucket mediante `hashCode()`:** Java calcula `s2.hashCode()`, obtiene 2236, y aplica una función de dispersión adicional para determinar el índice del bucket en el arreglo interno. Como `s1.hashCode()` también es 2236, **ambas caen en el mismo bucket**.
- **Colisión detectada:** El `HashSet` encuentra que ese bucket ya no está vacío (ya contiene a `s1`).
- **Verificación con `equals()`:** En lugar de asumir que son iguales por tener el mismo hash, Java recorre las entradas existentes en ese bucket y llama a `s2.equals(s1)` (es decir, `"Ea".equals("FB")`).
- **Resultado de `equals()`:** `"Ea".equals("FB")` devuelve **false**, porque el contenido de las cadenas es distinto carácter por carácter (aunque su combinación de valores ASCII/Unicode produzca la misma suma ponderada en la fórmula del `hashCode()` de `String`).

- **Conclusión:** Como `equals()` devuelve `false`, Java determina que `s2` es una clave **diferente**, no un duplicado. Por lo tanto, `s2` **se agrega** al mismo bucket que `s1` (como una nueva entrada en la lista enlazada/árbol de ese bucket), y el `HashSet` termina conteniendo **ambos elementos**.

`hashCode()` se usa para decidir **dónde buscar** (ubicar el bucket), pero `equals()` es el método que decide **si dos objetos son realmente el mismo elemento**. Un mismo hash no implica igualdad; es solo una condición *necesaria pero no suficiente* para que dos objetos sean considerados iguales en una colección basada en hash.

Si `a.equals(b) == true`, entonces obligatoriamente `a.hashCode() == b.hashCode()`.
Pero no es cierto lo inverso: `a.hashCode() == b.hashCode()` no implica que `a.equals(b) == true`.

2. En el **caso ideal**, un `HashMap` ofrece complejidad **$O(1)$** para `put()`, `get()` y `containsKey()`, porque la función hash distribuye las claves de manera uniforme entre los distintos buckets, de modo que cada bucket contiene, en promedio, muy pocos elementos (idealmente 0 o 1).

Sin embargo, en el **peor escenario posible**, esto se degrada de la siguiente manera:

Si muchas claves distintas producen el **mismo `hashCode()`** (o el mismo índice de bucket tras la función de dispersión interna), todas esas entradas terminan acumulándose en un **único bucket**. En ese caso, la estructura interna dentro del bucket deja de comportarse como una tabla hash y pasa a comportarse como una simple **lista enlazada** (o árbol, según la versión de Java).